MODULE 10

Error handling, optimistic rollback

ProblemDetails (RFC 7807) · errorInterceptor · snapshot rollback · 409 · E2E sprint

TMS resilient writes

```
builder.Services.AddProblemDetails();

app.UseStatusCodePages();


const detail = err.error?.detail;

patchState(store,
setAllEntities(snapshot));
```

Server errors and resilient UI

Session outcomes

By the end of this session, you will:

- Configure ASP.NET Core ProblemDetails (AddProblemDetails, UseStatusCodePages)
- Build a functional errorInterceptor in Angular to parse RFC 7807 JSON
- Implement optimistic deletion with automatic snapshot rollback (setAllEntities(snapshot))
- Perform the End-to-End Integration Sprint across CORS, HttpOnly cookies, XSRF, errors, and WebSockets

The optimistic sync cycle

- Snapshot first: capture store.entities() before any patchState you must be able to undo
- Mutate instantly: removeEntity or updateEntity so the UI feels responsive
- Roll back cleanly: on 4xx/5xx, restore the exact snapshot and surface ProblemDetails.detail

Flow in prose

Admin deletes course -> removeEntity (card vanishes) -> DELETE in background -> 204: done; 409/400: setAllEntities(snapshot) restores the row, errorInterceptor surfaces ProblemDetails.detail in a SnackBar.

Server ProblemDetails (RFC 7807)

Program.cs

```
// Program.cs  .NET 10 RFC 7807 setup

builder.Services.AddProblemDetails();

app.UseStatusCodePages();
```

- AddProblemDetails() turns model-state and 4xx responses into a standard JSON shape with type, title, status, detail
- UseStatusCodePages() ensures body-less status codes still get a ProblemDetails payload
- Angular reads err.error?.detail and falls back to a status-code map when detail is missing

Central errorInterceptor

errorInterceptor

```
// error.interceptor.ts  central error handler

export const errorInterceptor: HttpInterceptorFn = (req, next) => {

  const router = inject(Router);

  return next(req).pipe(

    catchError((err: HttpErrorResponse) => {

      const detail = err.error?.detail ?? 'A system error occurred.';

      if (err.status === 401) {

        router.navigate(['/login']);

      } else {
        console.error('API Error Payload:', detail);

      }
      return throwError(() => err);
    }),
  );
};
```

- Single place for HTTP error UX 401 to /login, everything else logs detail; SnackBar mapping lives in the consuming effect
- throwError rethrows so downstream error handlers (e.g. optimistic rollback) still see the error

Optimistic delete with snapshot rollback

removeCourse (trimmed)

```
// course.store.ts  optimistic delete (trimmed)
removeCourse(id: string) {
  const snapshot = store.entities();
  patchState(store, removeEntity(id));

  svc.deleteCourse(id).pipe(
    catchError((err) => {
      patchState(store, setAllEntities(snapshot));
      snack.open('Delete failed. Data restored.', 'OK');
      return throwError(() => err);
    })
  ).subscribe();
}
```

- Snapshot is captured BEFORE removeEntity the wrong order is the most common optimistic-UI bug
- On failure, setAllEntities(snapshot) restores the exact row state and SnackBar tells the admin why

409 Conflict and concurrency

CourseController (trimmed)

```
// CourseController.cs  version mismatch (trimmed)

[HttpPut("{id}")]

public async Task<IActionResult> Update(string id, CourseDto dto) {

  var course = await _db.Courses.FindAsync(id);

  if (course!.Version != dto.Version)

    return Conflict("This course was updated by another user.");

  // ... persist ...
}
```

- Admin edits while an instructor updates the same row -> 409 optimistic UI must roll back and show a clear conflict message
- Surface ProblemDetails.detail (server-controlled copy) so the message stays honest across releases
- When NOT to use optimistic UI: enrollment in a nearly-full course, financial calculations, irreversible deletes (use a confirm dialog)

Common mistakes Session 3

Mistake	Symptom	Fix
Snapshot after patchState (wrong order)	Rollback restores the wrong state	Capture store.entities() BEFORE removeEntity
Using err.message instead of err.error?.detail	Toast shows "Unprocessable Entity" or [object Object]	Use ProblemDetails.detail when the API returns it
EMPTY from catchError without rethrow	Downstream handlers never see the error	throwError(() => err) so rollback can run
Missing AddProblemDetails() on the API	Angular receives generic status text only	Add AddProblemDetails() and UseStatusCodePages() in Program.cs
Optimistic UI on near-full enrollment	Flash-and-rollback feels buggy	Loading spinner for high-rejection writes
No user-visible error	Blank screen on failure	Surface detail or mapped status in SnackBar / toast

Session 3 summary

- ProblemDetails (RFC 7807) standardizes C# server errors into clear client JSON payloads
- Functional interceptors centralize 401 redirects and error toast notifications
- Optimistic UI with snapshots gives instant visual feedback while remaining safe against server rejection
- 409 Conflict needs rollback plus clear someone-else-changed-this copy
- Full-stack fused: CORS, HttpOnly cookies, XSRF headers, error interceptors, and SignalR live updates work together under real browser rules

Interview angle

Walk an optimistic delete/update: what you show first, how you rollback on failure, and how you surface 409 from the same TMS concurrency rules as the API.

Next: Module 11 — Testing and QA for TMS

Unit, integration, and e2e nets on enrollments, APIs, and Angular flows you already built.